



INNOVATION & ENTREPRENEURSHIP HUB FOR EDUCATED RURAL YOUTH

SURE Trust – IERY

PROJECT REPORT

DebugAssist: Intelligent Bug Diagnosis & Resolution

Domain: Generative Artificial Intelligence

Project Period: March 2026 to September 2026

Mentor

Mr. Prujith Radhakrishna — Mentor, SURE Trust

Team Member

Ms. M. Geetha Sree

Project Information

Particular	Details
Project Title	DebugAssist: Intelligent Bug Diagnosis & Resolution
Domain	Generative AI
Organization	SURE Trust – Innovation & Entrepreneurship Hub for Educated Rural Youth (IERY)
Mentor	Mr. Prujith Radhakrishna — Mentor, SURE Trust
Team Member	Ms. M. Geetha Sree
Project Period	March 2026 to September 2026
Primary Application	AI-assisted software bug diagnosis and fix recommendation
User Interface	Streamlit
Core AI Approach	Multi-agent LLM pipeline with Retrieval-Augmented Generation

Declaration

The project titled “DebugAssist: Intelligent Bug Diagnosis & Resolution” has been mentored by Mr. Prujith Radhakrishna, organised by SURE Trust, from March 2026 to September 2026, for the benefit of educated unemployed rural youth for gaining hands-on experience in working on industry-relevant projects that would take them closer to prospective employers.

I declare that, to the best of my knowledge, the team member mentioned below has worked on the project successfully and enhanced her practical knowledge in the domain of Generative AI. The report presents the project objectives, implementation approach, evaluation results, learning outcomes, limitations, and possible future improvements.

Team Member: Ms. M. Geetha Sree

Mentor: Mr. Prujith Radhakrishna

Designation: Mentor — SURE Trust

Prof. Radhakumari
Executive Director & Founder
SURE Trust

Table of Contents

1. Executive Summary
2. Introduction
3. Project Objectives
4. Methodology & Results
5. Social / Industry Relevance of the Project
6. Learning & Reflection
7. Future Scope & Conclusion
8. Project Deliverables and Validation Summary
9. Appendix: Screenshot and Evidence Placement Guide

1. Executive Summary

Software development teams receive a continuous stream of bug reports, stack traces, error logs, and issue descriptions. Before an engineer can implement a correction, the incoming issue generally has to be understood, classified, compared with earlier defects, investigated for a probable root cause, and converted into an actionable remediation plan. These activities can consume considerable engineering time, particularly when historical defect information is distributed across large repositories and closed tickets.

DebugAssist is a Generative AI-based platform developed to support this first stage of software defect diagnosis. The system accepts a raw bug report, stack trace, or error log and processes it through a structured multi-agent workflow. Instead of depending only on an LLM's internal knowledge, the project uses Retrieval-Augmented Generation (RAG) to retrieve relevant historical defects from a defect knowledge base and provide that evidence to the reasoning stage.

The implemented workflow contains specialized stages for triage, log analysis, retrieval and root-cause reasoning, duplicate detection, and remediation. Each stage has a focused responsibility and produces structured information for the subsequent stage. This separation makes the overall workflow easier to inspect, evaluate, and improve than a single large prompt attempting to solve every task simultaneously.

The historical knowledge base contains more than 56,000 records aggregated from public Mozilla, Apache, and Eclipse bug repositories. Bug text is converted into embeddings using the sentence-transformers model all-MiniLM-L6-v2. Similar historical records are identified using cosine similarity through scikit-learn. The retrieved evidence is then supplied to the appropriate reasoning stage so that recommendations remain connected to previously observed defects.

A major innovation of the project is the feedback mechanism for knowledge-base growth. When a user confirms that a recommended fix was successful, the corresponding information can be embedded and appended to the retrieval corpus. This creates a practical feedback loop in which successful resolutions become future evidence instead of being lost after the current debugging session.

Validation results demonstrated 60% exact-match and 83.3% adjacent-level accuracy for the Triage Agent on a stratified 30-bug sample. The Log Analysis Agent achieved 81.8% fair-match accuracy for structured error-type extraction across 11 programming languages. In end-to-end testing, the system produced zero hallucinated fixes and achieved a 68.75% exact-fix-match rate across 16 fully completed test cases. Two confirmed-successful fixes also increased the knowledge-base record count from 56,862 to 56,864.

Overall, the project demonstrates practical experience in Generative AI application development, prompt engineering, embeddings, semantic retrieval, multi-agent orchestration, structured LLM outputs, evaluation, error handling, data processing, and Streamlit-based application development. It also emphasizes responsible reporting of system limitations rather than presenting only successful cases.

2. Introduction

2.1 Background and Context

Software defects are an unavoidable part of the software development lifecycle. A defect may appear as a compilation error, runtime exception, incorrect output, performance issue, integration failure, or unexpected application behaviour. In a professional engineering environment, a bug report is more than a short description of a problem. It can contain logs, stack traces, reproduction steps, affected modules, environment information, screenshots, and references to earlier issues.

The first response to such a report is often diagnostic rather than corrective. Engineers need to determine how urgent the issue is, what component may be affected, what type of failure occurred, whether a similar defect has already been reported, and what evidence can guide the next debugging step. When an organization has a large history of bug reports, that history becomes valuable institutional knowledge.

However, historical information is useful only when it can be retrieved efficiently. A conventional keyword search may fail when two defects describe the same underlying problem using different terminology. For example, an issue may describe an exception using one wording while an older ticket uses a different phrase but refers to the same failure mechanism. Semantic embeddings and similarity search provide a way to compare the meaning of text rather than relying only on exact words.

DebugAssist was therefore designed as an applied Generative AI project combining LLM reasoning with retrieval from historical software defects. The goal is not to replace software engineers. Instead, the platform is intended to accelerate the repetitive first-pass investigation and provide engineers with structured, evidence-backed information that can be reviewed before a final decision is made.

2.2 Problem Statement

The central problem addressed by the project is the difficulty of converting an unstructured software bug report into a useful, evidence-backed diagnosis. Manual processing can be slow and may vary between engineers. In addition, a large historical repository of resolved defects may not be fully utilized when engineers do not have an efficient way to discover semantically similar cases.

The project addresses the following related challenges:

- Classifying the severity and priority of incoming bugs in a consistent manner.
- Extracting useful diagnostic information from messy error messages and stack traces.
- Finding semantically similar historical defects rather than depending only on keyword matches.
- Identifying possible duplicate submissions among live bug reports.
- Producing a structured remediation recommendation based on available evidence.
- Reducing unsupported or hallucinated fix suggestions by grounding reasoning in retrieved historical information.
- Allowing confirmed successful fixes to contribute to the future knowledge base.

2.3 Scope of the Project

The project covers the complete diagnostic workflow from bug submission to fix recommendation and knowledge-base growth. The implementation is built in Python and exposed through a Streamlit application.

The scope includes data preparation, embedding-based retrieval, LLM-agent orchestration, duplicate similarity checking, structured output handling, validation, and analytics.

The project does not claim to autonomously modify source code, deploy fixes to production, or replace human approval. The remediation stage produces a recommendation for engineering review. This distinction is important because a software fix can have consequences beyond the text of a bug report, including compatibility, security, performance, and business requirements.

2.4 Limitations

- The system depends on an external, rate-limited LLM API; therefore, some evaluation cases could not be completed when the external quota was exhausted.
- Historical grounding is weaker for languages that are less represented in the source datasets, particularly Rust, Go, and TypeScript.
- Bug reports describing multiple unrelated problems at the same time may produce a compound classification instead of cleanly separating the issues.
- End-to-end evaluation was limited by the number of fully completed test cases available under the external API quota.
- The system provides decision support and recommendations; final diagnosis and production changes still require human engineering judgement.

2.5 Innovation Component

The primary innovation is the self-improving knowledge-base mechanism. Traditional retrieval systems may remain static unless their source data is manually updated. DebugAssist introduces a feedback path in which a confirmed successful fix can be converted into a retrievable record. This creates a mechanism for preserving newly validated debugging knowledge.

A second innovation is the use of multiple specialized agents instead of one general-purpose prompt. Triage focuses on classification, Log Analysis focuses on structured extraction, Root Cause focuses on evidence-based reasoning, Duplicate Detection focuses on similarity among live submissions, and Remediation combines the upstream information into an actionable recommendation.

3. Project Objectives

3.1 Primary Objectives

Objective / Component	Detailed Objective
Triage Agent	Classify bug severity, priority, and affected component and provide a confidence score.
Log Analysis Agent	Extract structured diagnostic fields such as specific error type, failure location, and code path from messy multi-language error text.
Root Cause Agent	Use embedding-based retrieval of historical defects to generate a grounded root-cause hypothesis.
Duplicate Detection Agent	Identify likely duplicate bug submissions using cosine similarity over live submissions.
Remediation Agent	Combine upstream diagnostic outputs and retrieved evidence into a structured, actionable fix recommendation.
Knowledge-Base Growth	Add confirmed-successful fixes back into the retrieval corpus so that future searches can benefit from newly validated information.
Evaluation	Measure individual agent performance and the complete end-to-end pipeline using documented validation cases.

3.2 Expected Outcomes

- A working Streamlit application supporting bug submission, diagnosis, recommendations, and analytics.
- A historical defect knowledge base containing 56,000+ records.
- A semantic retrieval layer based on sentence-transformer embeddings and cosine similarity.
- A multi-agent diagnostic workflow with structured JSON outputs.
- Validation results for individual agents and the complete pipeline.
- A documented mechanism for growing the knowledge base through confirmed successful fixes.
- Project source code and documentation maintained in a GitHub repository.

3.3 Functional Requirements

The system should accept a textual bug description or error information and return structured diagnostic information. The user should be able to understand the major classification, extracted error details, relevant historical evidence, duplicate indication where applicable, and final remediation recommendation. The interface should present the information in a form that supports human review rather than exposing only a raw LLM response.

The retrieval component should return historically relevant records based on semantic similarity. The reasoning component should use retrieved evidence when forming a root-cause hypothesis and fix recommendation. The application should also maintain sufficient information for analytics and for the knowledge-base feedback mechanism.

3.4 Non-Functional Considerations

- Reliability: retry-with-backoff handling is used for rate-limited LLM calls.

- Interpretability: specialized agents and structured outputs make intermediate results easier to inspect.
- Maintainability: each major diagnostic responsibility is separated into a distinct processing stage.
- Scalability: future versions can migrate the current retrieval storage into a more scalable chunked or incremental format.
- Safety: retrieved historical evidence is used to reduce unsupported recommendations, while final engineering decisions remain with the user.

4. Methodology & Results

4.1 Overall Methodology

The project follows a pipeline-based methodology. A submitted bug first passes through triage and log analysis. The resulting structured information is then used to query the historical defect knowledge base. The retrieved records support root-cause reasoning. In parallel, the live submission collection can be checked for similarity to identify possible duplicates. The resulting information is finally combined by the Remediation Agent.

Each LLM-based stage is implemented as an independently prompted call with a fixed JSON output schema. This provides a predictable contract between stages. Retry-with-backoff handling is included to manage failures caused by external API rate limits. The approach also allows individual stages to be evaluated separately rather than evaluating only the final response.

4.2 End-to-End Processing Flow

1. Bug submission — the user provides a bug description, stack trace, or error log.
2. Triage — severity, priority, affected component, and confidence are determined.
3. Log analysis — error type, failure location, code path, and related diagnostic fields are extracted.
4. Semantic retrieval — the bug information is embedded and compared against historical defect records.
5. Root-cause reasoning — retrieved evidence is used to formulate a grounded hypothesis.
6. Duplicate detection — the incoming issue is compared with live submissions using cosine similarity.
7. Remediation — the upstream results and historical evidence are synthesized into a structured fix recommendation.
8. User review — the recommendation is displayed for engineering judgement and validation.
9. Feedback loop — when a recommendation is confirmed successful, the new knowledge can be embedded and appended to the retrieval corpus.

4.3 Technology Stack

Technology / Tool	Role in the Project
Python	Primary implementation language for data processing, orchestration, retrieval, and application logic.
Streamlit	User interface for bug submission, results presentation, and analytics.
Groq API / Llama 3.3 70B Versatile	LLM reasoning for the specialized agent stages.
sentence-transformers	Generation of semantic embeddings using all-MiniLM-L6-v2.
scikit-learn	Cosine similarity and supporting machine-learning utilities.
LangChain	Text chunking and supporting retrieval-oriented processing.
pandas / NumPy	Data preparation, manipulation, and numerical processing.
SQLite	Storage of live submissions.

Plotly	Analytics and visualization dashboard.
--------	--

4.4 Data Collection and Preparation

The historical knowledge base aggregates public bug-tracking datasets from the Mozilla, Apache, and Eclipse projects through Kaggle. These records provide historical examples of software defects and their associated descriptions. A separate labeled multi-language stack-trace dataset covering 11 programming languages was used to validate the Log Analysis Agent.

The historical records form the evidence layer for the RAG pipeline. Rather than sending the entire knowledge base to the LLM, the system creates semantic representations of the available defect information and retrieves only the records most relevant to the current query. This reduces the amount of information that must be passed into the reasoning stage and creates a direct connection between the recommendation and historical evidence.

4.5 Embedding and Semantic Retrieval

For semantic retrieval, bug text is converted into an embedding using the sentence-transformers model all-MiniLM-L6-v2. An embedding represents the input text as a numerical vector in a semantic space. Texts with related meanings can therefore be compared through vector similarity even when their exact wording differs.

The project uses cosine similarity through scikit-learn to compare the current bug representation with historical defect representations. The highest-relevance records are retrieved and supplied to the root-cause reasoning stage. This retrieval process is the core RAG mechanism of the application: retrieval provides external project-specific evidence, while the LLM uses that evidence to produce a human-readable diagnosis.

This approach is preferable to unconstrained generation for the project's use case because a debugging recommendation should ideally be supported by previously observed defect information. The retrieval layer does not guarantee correctness, but it gives the reasoning stage concrete evidence to consider and reduces dependence on unsupported model memory.

4.6 Multi-Agent Architecture

The architecture is organized around specialized responsibilities. The Triage Agent answers the question of how urgent and severe the issue appears. The Log Analysis Agent converts unstructured error text into structured fields. The retrieval stage finds related historical defects. The Root Cause Agent interprets the retrieved evidence and proposes a likely explanation. Duplicate Detection compares the issue against live submissions. The Remediation Agent then combines these outputs.

The use of structured JSON contracts is important because downstream components should receive predictable fields rather than arbitrary natural-language responses. It also makes evaluation easier: individual fields can be inspected and compared against expected results. Retry-with-backoff handling provides resilience when external model requests fail temporarily or encounter rate limits.

4.7 Architecture Diagram — Placement

Insert the final architecture screenshot/diagram in this section. The diagram should visually show the following flow:

User Bug Report → Triage Agent + Log Analysis Agent → Embedding / Semantic Retrieval → Historical Defect Knowledge Base → Root Cause Agent → Remediation Agent → User Recommendation.

In parallel, the User Bug Report is compared against Live Submissions → Duplicate Detection Agent → Duplicate Result → Remediation stage. A confirmed successful fix flows into the Knowledge-Base Growth mechanism → embedding → retrieval corpus.

Suggested figure caption: Figure 1. High-level architecture and feedback loop of the DebugAssist diagnostic platform.

4.8 Results and Validation

Component	Validation Setup	Result
Triage Agent	Stratified sample of 30 bugs	60% exact-match; 83.3% adjacent-level accuracy
Log Analysis Agent	55-bug sample across 11 languages	81.8% fair-match accuracy
End-to-End Pipeline	16 fully completed cases out of 39 sampled	0 hallucinated fixes; 68.75% exact-fix-match
Knowledge-Base Growth	Two confirmed-successful fixes	56,862 → 56,864 records

The results indicate that the system can perform useful structured diagnosis while also demonstrating the importance of evaluation methodology. For the Triage Agent, exact agreement was lower than adjacent-level agreement, which means some predictions were close to the expected severity or priority even when they did not match exactly. This distinction is useful when interpreting LLM classification performance.

For Log Analysis, the reported fair-match metric accounts for the fact that semantically equivalent error descriptions may not always be represented using exactly the same string. This provides a more meaningful view of extraction quality than relying exclusively on exact string comparison.

The end-to-end result is especially relevant to the project's central objective. Across the fully completed test cases, no hallucinated fixes were observed and 68.75% achieved an exact fix match. However, the evaluation was constrained by external API quota, with the remaining sampled cases not fully completed. Therefore, the result should be interpreted as a documented validation outcome rather than a universal accuracy guarantee.

4.9 Known Limitations Observed During Validation

- Rate limits on the external LLM API prevented completion of all sampled end-to-end cases.
- Languages with fewer historical records have weaker retrieval grounding.
- Compound bug reports can create ambiguous classifications.
- Semantic retrieval quality depends on the coverage and quality of the historical knowledge base.
- A high similarity score does not by itself prove that two defects have the same root cause.

4.10 Final Working Screenshots

Dashboard interface screenshot below:

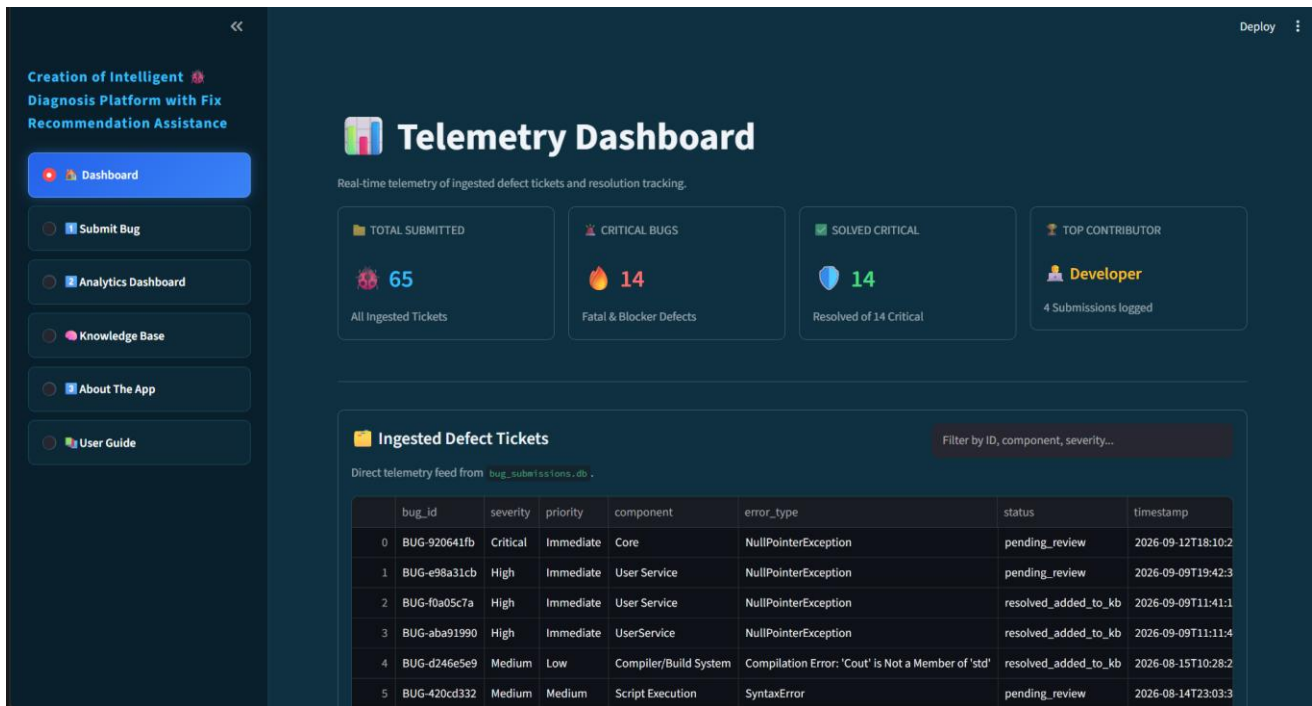
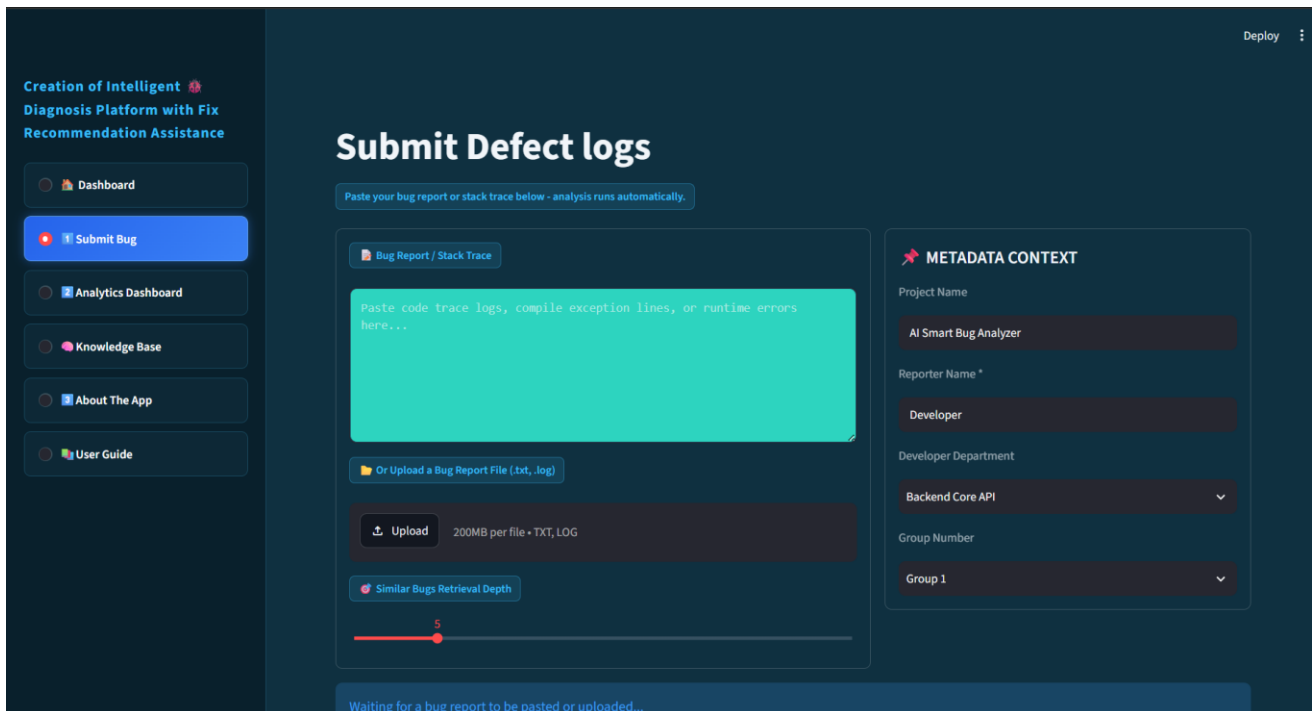


Figure 2. Bug submission interface — insert screenshot here.



Supporting explanation: This screenshot should be accompanied by 2–4 sentences explaining what the user sees, which component produced the result, and how the displayed information supports the debugging workflow.

Figure 3. Structured diagnostic findings — insert screenshot here.

Analysis Summary

Severity	Priority	Component
Critical	Immediate	Core

Error Type: NullPointerException

Failure Location: Main.java:6

Root Cause (summary): The NullPointerException likely occurs because a String variable is null when Main.main() calls String.toUpperCase(), mirroring past cases where null ...

Duplicates Found: None — this appears to be a new issue

Recommended Fix (summary): Add a null-check (or default value) before invoking String.toUpperCase() and ensure the variable is properly initialized earlier in the code.

Show full details (all agents)

Supporting explanation: This screenshot should be accompanied by 2–4 sentences explaining what the user sees, which component produced the result, and how the displayed information supports the debugging workflow.

Figure 4. Root-cause / historical evidence display — insert screenshot here.

Root Cause Analysis

Hypothesis: The `NullPointerException` likely occurs because a `String` variable is null when `Main.main()` calls `String.toUpperCase()`, mirroring past cases where null arguments were passed without validation, leading to NPEs in core utility code.

Confidence: 0.84

Supporting Evidence:

- `WTP_SOURCE_EDITING-325879` — NPE triggered in core helper when processing null input
- `WSCOMMONS-2` — `IllegalArgumentException` caused by passing null to constructor

Full Combined Result (Triage + Log Analysis)

```
{
  "bug_id": "BUG-920641fb"
  "triage": {
    "bug_id": "BUG-920641fb"
    "severity": "Critical"
    "confidence": 0.93
    "reasoning":
      "The application throws a NullPointerException in the main thread, causing an immediate crash and preventing any functionality from running."
    "priority": "Immediate"
    "component": "Core"
  }
  "log_analysis": {
    "error_type": "NullPointerException"
    "failure_location": "Main.java:6"
    "code_path": "main thread -> Main.main() -> String.toUpperCase()"
    "confidence": 0.99
    "reasoning":
      "A variable (referenced as <local5>) is null, so invoking its toUpperCase() method causes a NullPointerException at the indicated line."
  }
}
```

Supporting explanation: This screenshot should be accompanied by 2–4 sentences explaining what the user sees, which component produced the result, and how the displayed information supports the debugging workflow.

Figure 5. Duplicate detection result — insert screenshot here.

Duplicate Bugs

No similar past submissions found — this appears to be a new issue.

Full Duplicate Detection Result

▶ []

Supporting explanation: This screenshot should be accompanied by 2–4 sentences explaining what the user sees, which component produced the result, and how the displayed information supports the debugging workflow.

Figure 6. Remediation recommendation — insert screenshot here.

The screenshot displays the DebugAssist application interface. On the left is a sidebar with navigation links: "Creation of Intelligent Diagnosis Platform with Fix Recommendation Assistance", "Dashboard", "Submit Bug" (highlighted in blue), "Analytics Dashboard", "Knowledge Base", "About The App", and "User Guide". The main content area is titled "Recommended Fix" and includes a "Deploy" button in the top right corner. The text states: "Add a null-check (or default value) before invoking String.toUpperCase() and ensure the variable is properly initialized earlier in the code." Below this, the "Confidence" is listed as 0.94. The "Fix Steps" are numbered 1 through 5, detailing the process from locating the variable to re-compiling and running the program. A comparison between "Before" and "After" code snippets is shown, illustrating the addition of a null check. Finally, "Validation Steps" are listed, including running the application with the original test case, adding a unit test for null input, executing the existing test suite, and checking logs for warnings.

Recommended Fix

Add a null-check (or default value) before invoking `String.toUpperCase()` and ensure the variable is properly initialized earlier in the code.

Confidence: 0.94

Fix Steps:

1. Locate the variable that is passed to `toUpperCase()` in `Main.main()` (e.g., the 'input' string).
2. Determine why it can be null – either it is not assigned, or it comes from an external source that may be missing.
3. Add a guard clause or default assignment before the call, e.g., `if (input == null) { input = ""; }`
4. Alternatively, use a conditional expression when calling `toUpperCase()`: `String result = (input != null) ? input.toUpperCase() : "";`
5. Re-compile and run the program to verify the `NullPointerException` no longer occurs.

Before:

```
String result = input.toUpperCase();
```

After:

```
String result = (input != null) ? input.toUpperCase() : "";
```

Validation Steps:

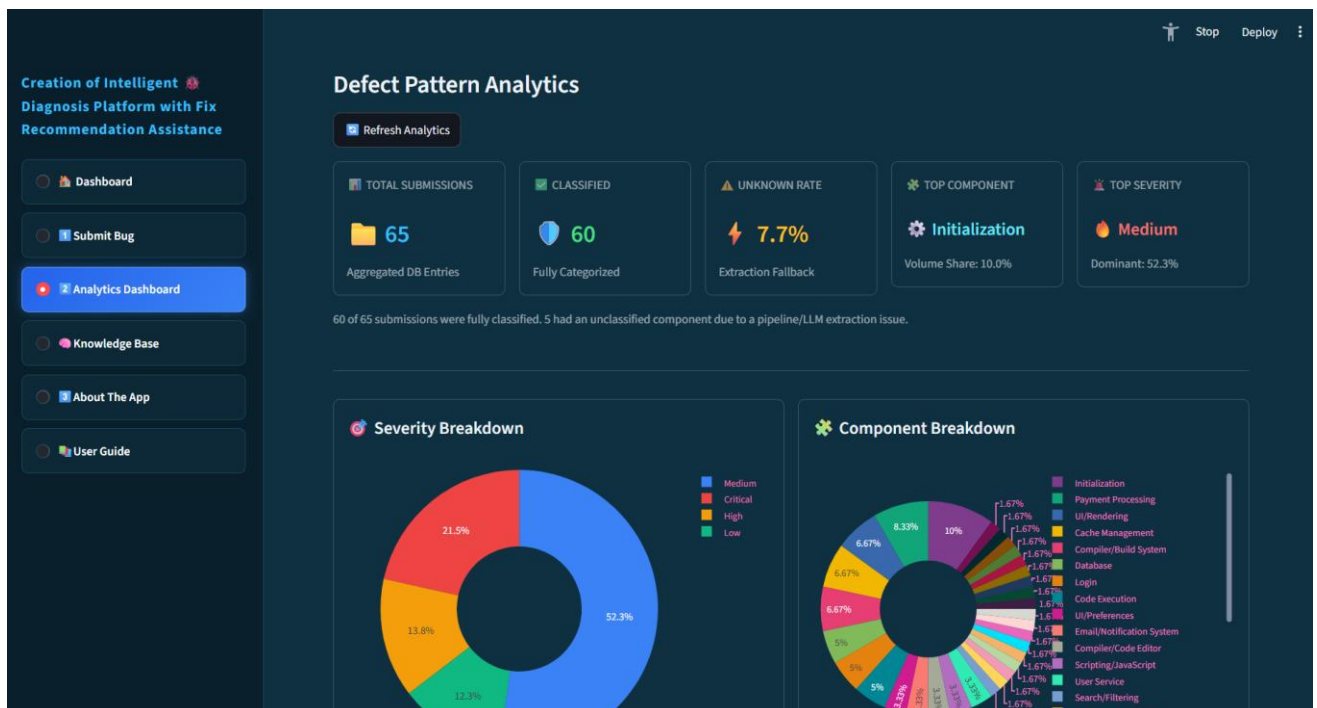
- Run the application with the original test case that caused the NPE and confirm it now completes without exception.
- Add a unit test that passes a null string to the method and asserts the returned value is the expected default (e.g., empty string).
- Execute existing test suite to ensure no regressions in other parts of the core component.
- Check logs for any new warnings about unexpected null inputs.

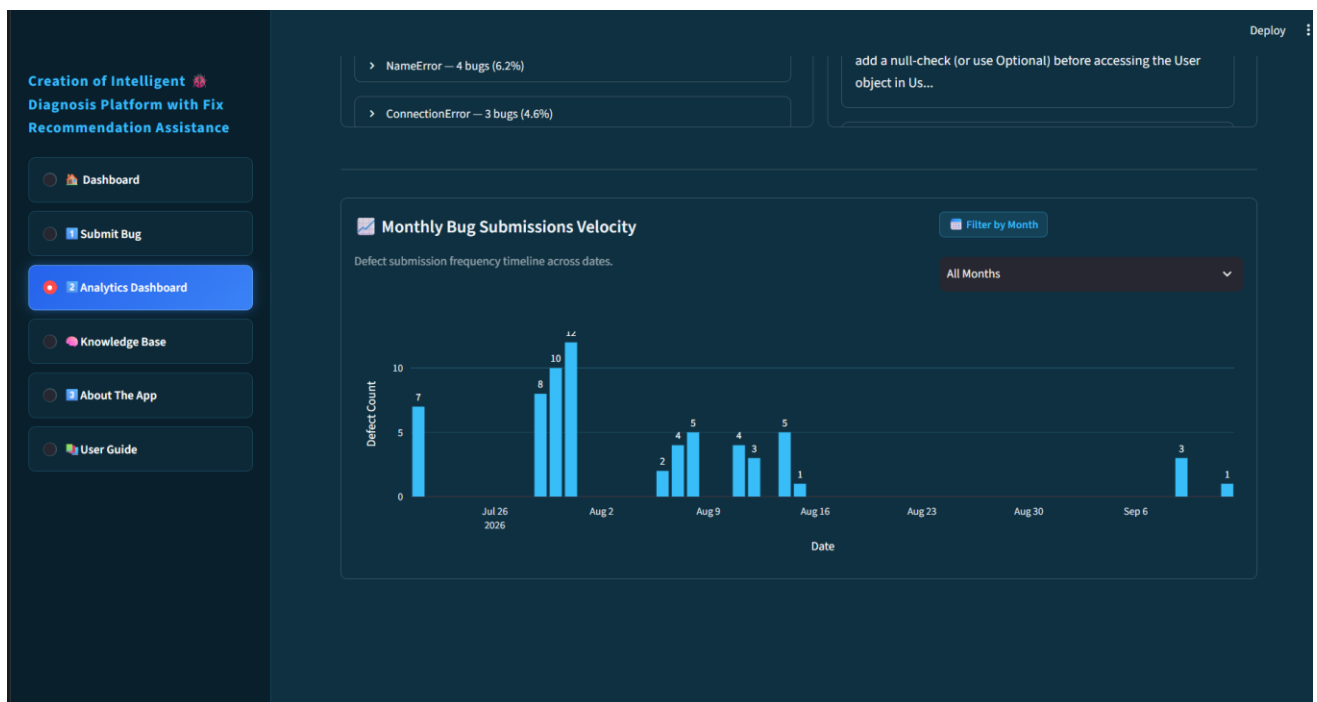
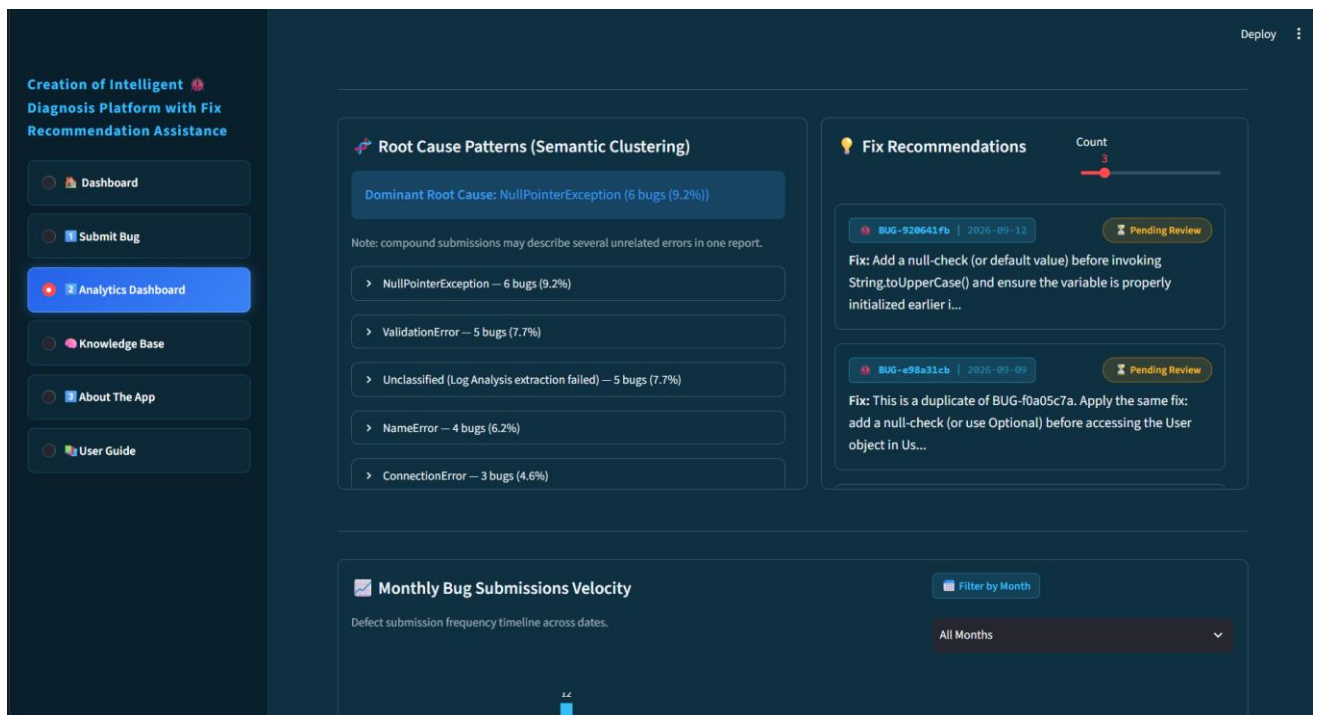
Full Remediation Result

```
{
  "bug_id": "BUG-920641fb"
  "recommended_fix":
    "Add a null-check (or default value) before invoking String.toUpperCase() and ensure the variable is properly initialized earlier in the code."
  "fix_steps": [
    0: "Locate the variable that is passed to toUpperCase() in Main.main() (e.g., the 'input' string)."
    1:
      "Determine why it can be null – either it is not assigned, or it comes from an external source that may be missing."
    2: "Add a guard clause or default assignment before the call, e.g., if (input == null) { input = \"\"; }"
    3:
      "Alternatively, use a conditional expression when calling toUpperCase(): String result = (input != null) ? input.toUpperCase() : \"\";"
    4: "Re-compile and run the program to verify the NullPointerException no longer occurs."
  ]
  "code_example": {
    "before": "String result = input.toUpperCase();"
    "after": "String result = (input != null) ? input.toUpperCase() : \"\";"
  }
  "validation_steps": [
    0:
      "Run the application with the original test case that caused the NPE and confirm it now completes without exception."
    1:
      "Add a unit test that passes a null string to the method and asserts the returned value is the expected default (e.g., empty string)."
    2: "Execute existing test suite to ensure no regressions in other parts of the core component."
    3: "Check logs for any new warnings about unexpected null inputs."
  ]
}
```

Supporting explanation: This screenshot should be accompanied by 2–4 sentences explaining what the user sees, which component produced the result, and how the displayed information supports the debugging workflow.

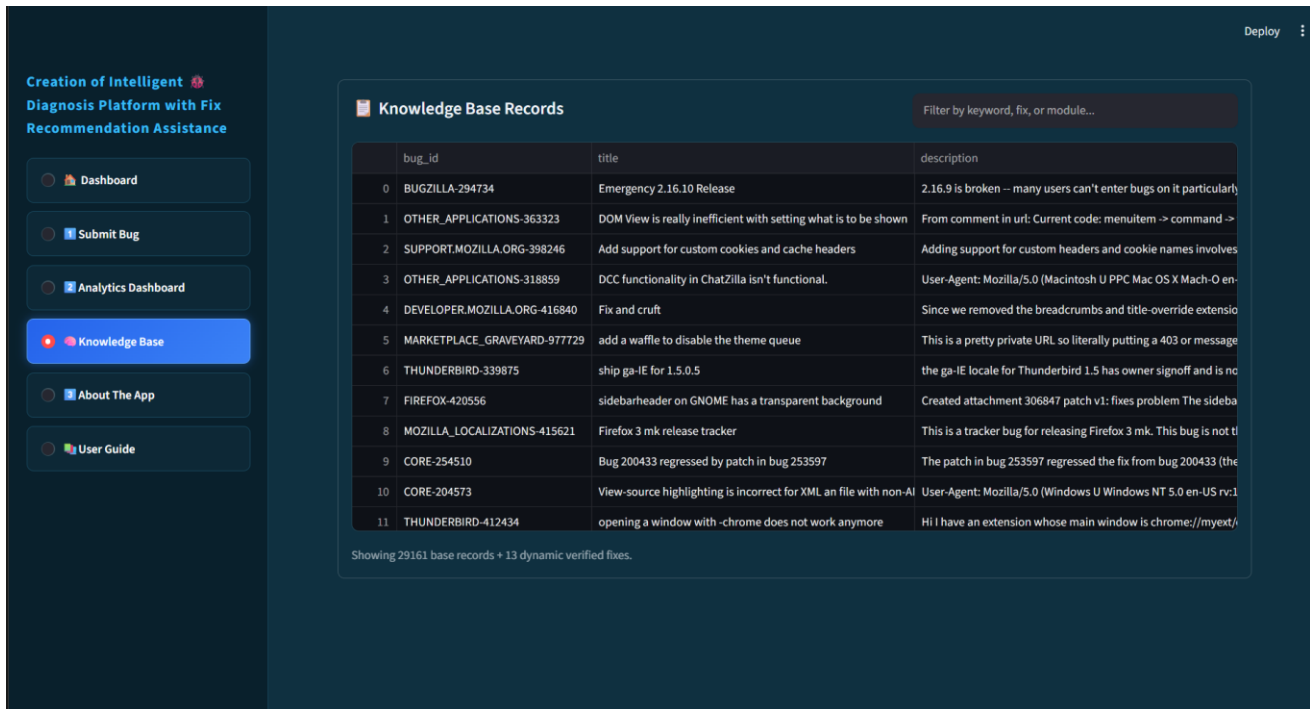
Figure 7. Analytics dashboard — insert screenshot here.



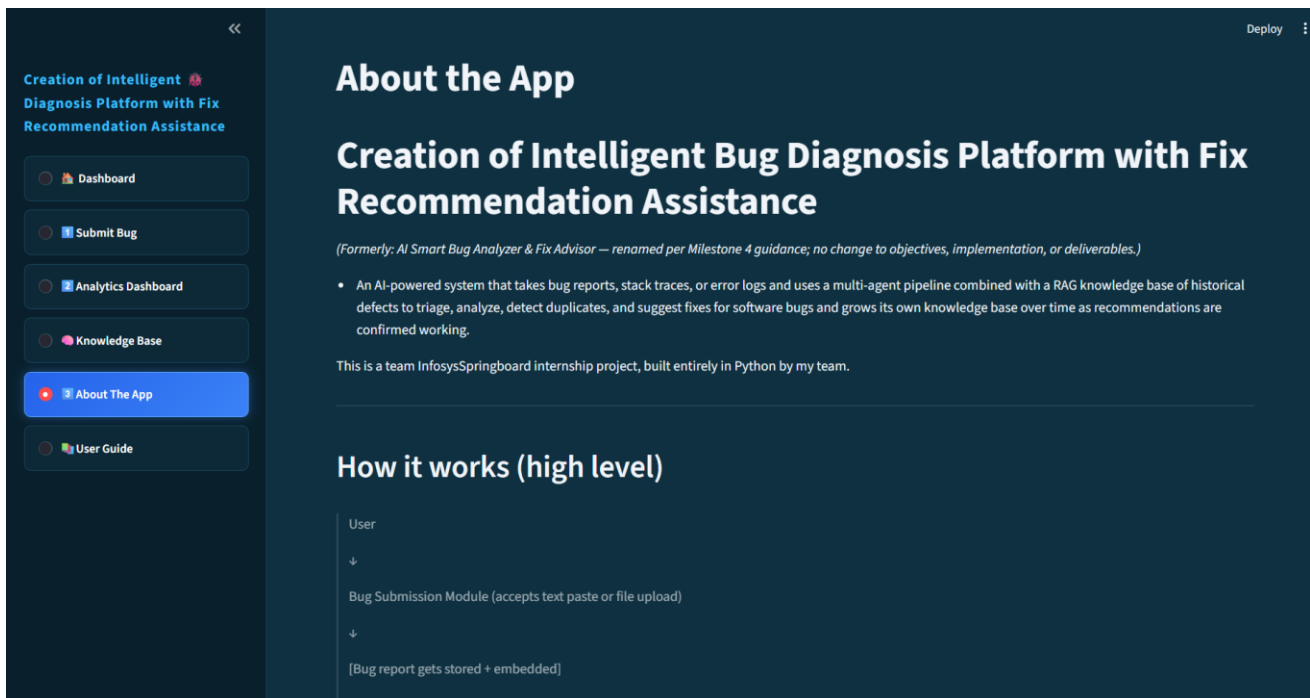


Supporting explanation: This screenshot should be accompanied by 2–4 sentences explaining what the user sees, which component produced the result, and how the displayed information supports the debugging workflow.

Knowledge base data is below screenshot



About the app-screenshot inserted below



4.11 Project GitHub Link

[Insert the final SURE Trust GitHub repository link here.]

5. Social / Industry Relevance of the Project

5.1 Industry Relevance

Modern software organizations manage large numbers of defects across multiple products, services, repositories, and teams. The initial investigation of a bug often contains repetitive work: understanding the issue, classifying urgency, locating related incidents, and searching historical records. Automating part of this process can help engineers spend more time on deeper technical investigation and implementation.

DebugAssist is relevant because it combines Generative AI with a concrete enterprise engineering workflow rather than using an LLM only as a general-purpose chatbot. The system demonstrates how an organization could connect an LLM to its own historical knowledge and use retrieval to make the output more context-specific.

The use of RAG is particularly relevant to enterprise environments because internal knowledge is often not present in a public model's training data. A retrieval layer provides a mechanism to bring organization-specific evidence into the reasoning process. In this project, the evidence is historical defect data from Mozilla, Apache, and Eclipse repositories.

5.2 Reduction of Repetitive Engineering Effort

Bug triage and first-pass diagnosis can consume substantial engineering time. DebugAssist targets the repetitive portions of this process by structuring incoming information and presenting historical evidence. It does not remove the need for engineers; rather, it aims to reduce the time required to reach an informed starting point.

A useful outcome of such a system is consistency. Two engineers reviewing similar reports can receive similarly structured diagnostic fields and evidence, making the initial workflow more standardized. The final decision can still remain with the responsible engineering team.

5.3 Addressing AI Hallucination Risk

One of the major concerns with applying Generative AI to technical workflows is hallucination: the model may produce a plausible-looking answer that is not supported by evidence. DebugAssist addresses this concern at the architecture level by introducing a retrieval step before root-cause and remediation reasoning.

The project's end-to-end validation reported zero hallucinated fixes in the fully completed test set. This is a useful validation observation, but it should not be interpreted as proof that hallucinations are impossible. Continued testing, stronger evaluation datasets, and human review are still necessary before using such a system in production-critical workflows.

5.4 Knowledge Preservation

Organizations often accumulate valuable debugging knowledge in closed tickets. Once an issue is resolved, the solution may be difficult to reuse unless another engineer searches for the old ticket. The knowledge-base growth mechanism in DebugAssist provides a structured direction for turning confirmed successful fixes into future retrieval evidence.

This feedback mechanism is particularly useful for long-running engineering environments. As the number of validated fixes grows, the system can potentially become more useful for recurring classes of defects. The

quality of the resulting knowledge base will depend on the quality of confirmations and the consistency of the stored records.

5.5 Relevance to Educated Rural Youth

The SURE Trust IERY programme is intended to provide educated unemployed rural youth with practical exposure to industry-relevant projects. DebugAssist provides experience across several layers of modern technology: data processing, Generative AI, embeddings, semantic search, prompt engineering, application development, evaluation, and technical documentation.

The project therefore supports employability by converting theoretical concepts into a complete working application. It also encourages an engineering mindset in which the developer measures performance, documents limitations, and considers how a prototype could be improved for real-world use.

6. Learning & Reflection

6.1 Technical Learning

Area	Learning / Reflection
Multi-agent LLM design	Learned how to divide a complex AI task into specialized stages with clearly defined responsibilities rather than depending on one monolithic prompt.
RAG	Learned how retrieval can provide external evidence to an LLM so that responses are grounded in a project-specific knowledge base.
Embeddings	Gained practical understanding of converting text into vector representations and using semantic similarity to identify related defect records.
Cosine similarity	Learned how vector similarity can support semantic search and duplicate detection.
Prompt engineering	Learned that prompts need iterative refinement based on observed errors, including severity calibration, error-type specificity, and structured JSON formatting.
Structured LLM outputs	Learned the value of fixed JSON schemas for passing reliable information between agents and simplifying evaluation.
API error handling	Learned to account for rate limits and temporary external-service failures using retry-with-backoff logic.
Data processing	Worked with public defect datasets and multi-language stack-trace data for retrieval and validation.
Streamlit development	Learned how to convert an AI pipeline into an interactive application and present intermediate and final results.
Evaluation	Learned to compare predictions with expected results and to distinguish exact matching from fair or semantically acceptable matching.

6.2 Project Management and Engineering Learning

- Breaking a large project into independent components made development and debugging more manageable.
- Defining the expected input and output of each agent reduced ambiguity between stages.
- Testing individual components before end-to-end testing helped isolate failures.
- Documenting limitations was treated as part of the engineering process rather than as an afterthought.
- External API limits demonstrated the importance of planning for dependency failures in AI applications.
- Evaluation results showed that a project should be judged using measurable outcomes rather than only visual demonstrations.

6.3 Overall Experience

Working on DebugAssist as part of the SURE Trust Generative AI internship provided hands-on experience in building an AI application from raw data through to a validated interface. The project required connecting multiple concepts that are often learned separately: data preparation, embeddings, semantic search, LLM prompting, agent orchestration, application development, persistence, error handling, and evaluation.

One of the most important experiences was learning to distinguish a prototype that produces impressive outputs from a system that can be evaluated. The project included explicit validation results and documented limitations. This encouraged a more realistic approach to Generative AI development, where performance, failure cases, data coverage, and external dependencies all influence the quality of the final system.

The project also improved problem-solving ability. When an output did not match expectations, the issue could originate from the prompt, retrieval evidence, similarity threshold, structured-output formatting, dataset limitations, or external API behaviour. Investigating these possibilities helped develop a more systematic debugging approach for AI systems themselves.

6.4 Professional Skills Developed

- Technical documentation and report preparation.
- Explaining an AI system through architecture, workflow, and evaluation metrics.
- Interpreting test results without overstating accuracy.
- Understanding trade-offs between automation and human review.
- Working with external APIs and handling operational constraints.
- Presenting technical work in a form that can be understood by both technical and non-technical stakeholders.

7. Future Scope & Conclusion

7.1 Future Scope

Future Area	Proposed Improvement
Automated evaluation harness	Build a repeatable evaluation framework that can execute larger numbers of test cases and automatically calculate agent-level and end-to-end metrics.
Scalable knowledge storage	Migrate the current retrieval storage to a chunked or incremental storage approach to improve scalability as the knowledge base grows.
Compound issue handling	Introduce explicit detection and decomposition of bug reports containing multiple unrelated issues.
Broader language coverage	Increase historical grounding for underrepresented languages such as Rust, Go, and TypeScript.
Improved feedback validation	Add stronger checks around user-confirmed fixes so that only sufficiently validated resolutions are added to the knowledge base.
Richer analytics	Extend the dashboard to track retrieval quality, common error categories, agent confidence, and recurring defect patterns.
Enterprise integration	A future version could connect with issue trackers and internal engineering knowledge sources, subject to organizational security and access requirements.
Human-in-the-loop workflow	Introduce explicit approval stages so engineers can accept, reject, edit, or annotate recommendations before knowledge is fed back into the retrieval corpus.

7.2 Conclusion

DebugAssist successfully demonstrates a Generative AI approach to software bug diagnosis by combining specialized LLM agents, semantic retrieval, historical defect evidence, duplicate detection, and structured remediation recommendations. The project covers the complete workflow from bug submission through diagnosis and recommendation, while also introducing a feedback mechanism for knowledge-base growth.

The implemented system uses Python, Streamlit, Groq API, sentence-transformers, scikit-learn, LangChain, pandas, NumPy, SQLite, and Plotly. The historical knowledge base contains more than 56,000 defect records from Mozilla, Apache, and Eclipse sources. Validation produced measurable results for triage, log analysis, and end-to-end diagnosis, including zero observed hallucinated fixes in the fully completed test cases.

The project also demonstrates that applied Generative AI requires more than selecting an LLM. A useful system needs data, retrieval, prompts, structured outputs, error handling, evaluation, and a clear understanding of limitations. The experience gained through DebugAssist therefore extends beyond one application and provides a foundation for developing reliable AI-assisted engineering tools.

In conclusion, the project achieved its core objective of building and validating a working AI-assisted bug diagnosis platform. With stronger evaluation, scalable storage, broader language coverage, and deeper enterprise integration, the system has scope to evolve into a more comprehensive engineering support solution.

8. Project Deliverables and Validation Summary

Deliverable	Status / Evidence to Include
Working Streamlit application	Implemented; include final application screenshots.
Multi-agent diagnostic pipeline	Implemented; include architecture diagram and workflow explanation.
Historical defect knowledge base	56,000+ records; document source datasets and final count.
Semantic retrieval layer	Implemented using sentence-transformers and cosine similarity.
Duplicate detection	Implemented using similarity over live submissions.
Remediation recommendation	Implemented as the final synthesis stage.
Knowledge-base growth	Verified increase from 56,862 to 56,864 after two confirmed-successful fixes.
Agent validation	Triage and Log Analysis results documented.
End-to-end validation	16 fully completed cases; 0 hallucinated fixes; 68.75% exact-fix-match.
GitHub source code	Insert final repository link.
Final screenshots	Insert bug submission, findings, evidence, recommendation, and analytics screenshots.

9. Appendix: Screenshot and Evidence Placement Guide

To make the final submission look like a complete project report rather than a short textual document, the following evidence should be inserted in the indicated sections. Each screenshot should have a figure number, descriptive caption, and a short explanation below it.

- Application home / bug submission page — demonstrate the input area and available actions.
- Triage result — show severity, priority, affected component, and confidence.
- Log analysis result — show extracted error type and other structured fields.
- Historical retrieval / root-cause evidence — show relevant historical records or evidence presented to the user.
- Duplicate detection — show the similarity-based duplicate indication for a test case.
- Remediation recommendation — show the final structured recommendation.
- Analytics dashboard — show the available project-level visualizations.
- Knowledge-base feedback — show evidence of a confirmed successful fix being added, if the application UI exposes this step.
- GitHub repository — insert the final repository URL and optionally a screenshot of the project structure.

Report preparation note: The current report content is intentionally expanded around the exact project information already documented. Any project-specific evidence that is not currently present—such as final screenshots, the final GitHub URL, or additional measured test cases—should be inserted rather than estimated.

Final Sign-off

Team Member: Ms. M. Geetha Sree

Signature: _____

Date: _____

Mentor: Mr. Prujith Radhakrishna

Signature: _____